

REQUIREMENT ANALYSIS AND CUSTOMER DATABASE MODULE

1. Requirement Analysis

Understanding E-Commerce Business:

An e-commerce application is designed to support online purchasing by connecting customers, sellers, products, orders, and payment activities. A well-structured database is required to store these details and maintain the relationship between different activities of the purchasing process.

The Process Flow:

- **Browse / Search Product:** Customers can explore the available products and search for an item based on their requirements.
- **Add to Cart:** After selecting a suitable product, the customer can add it to the shopping cart.
- **Order Placed:** The customer provides the required delivery information and confirms the order.
- **Checkout / Payment:** The order moves to the checkout stage, where the customer completes the payment using an available payment method.
- **Delivery:** After successful payment, the order is processed for dispatch and delivered to the customer's specified address.
- **Review & Support:** Once the product is delivered, the customer can provide feedback or request assistance for returns, replacements, installation, or other product-related issues.

Entities:

1. **Customer:** The Customer entity stores the basic information of users who purchase products through the e-commerce platform. Each customer is identified by a unique customer ID.

Key Attributes:

Customer_ID (PK), Customer_Name, Email, Phone, created_at

Key Relationships:

One-to-Many relationship with Order and Review.

2. **Seller:** The Seller entity contains information about vendors who offer products on the platform. A seller may provide multiple products.

Key Attributes:

seller_id (PK), company_name, email, phone, rating, bank_details

Key Relationships:

One-to-Many relationship with Product.

3. **Product:** The Product entity represents the items listed by sellers for customers to purchase. It contains information related to the product, price, stock, and category.

Key Attributes:

product_id (PK), seller_id (FK), product_name, description, price, stock_quantity

Key Relationships:

Many-to-One relationship with Seller and One-to-Many relationship with Review.

- 4. Order:** The Order entity maintains information about purchases made by customers. It records details such as the order date, status, amount, and delivery address.

Key Attributes:

order_id (PK), customer_id (FK), order_date, status, total_amount, shipping_address

Key Relationships:

Many-to-One with Customer, One-to-One with Payment, and Many-to-Many with Product through OrderItem.

- 5. Payment:** The Payment entity records the transaction information associated with an order. It helps maintain payment method, transaction status, amount, and payment date.

Key Attributes:

payment_id (PK), order_id (FK), payment_method, status, transaction_id, amount, payment_date

Key Relationships:

One-to-One relationship with Order.

- 6. Review:** The Review entity maintains the feedback given by customers for products. It includes the rating and comments provided after the product experience.

Key Attributes:

review_id (PK), customer_id (FK), product_id (FK), rating, comment, created_at

Key Relationships:

Many-to-One relationship with Customer and Product.

2. Customer Table Creation

The Customer table is created to maintain the details of customers using the e-commerce application. The customer ID acts as the unique identifier, while the other fields store the customer's personal and registration information.

```
SQL> CREATE TABLE CUSTOMER(customer_ID INT PRIMARY KEY, customer_Name VARCHAR(100) NOT NULL, Email VARCHAR(100) UNIQUE, Phone NUMBER(15) UNIQUE, Address VARCHAR(200), Registration_Date DATE NOT NULL);  
Table created.
```

Insert Values:

Customer records are added to the database using the INSERT statement. Each record contains the required information corresponding to the columns defined in the Customer table.

```

SQL> INSERT INTO CUSTOMER VALUES('101','Pooja','pooja@gmail.com',9876543225,'Madurai','15-MAR-26');
1 row created.

SQL> INSERT INTO CUSTOMER VALUES('102','Hari','hari@gmail.com',9876543226,'Salem','01-APR-26');
1 row created.

SQL> INSERT INTO CUSTOMER VALUES('103','Lakshmi','lakshmi@gmail.com',9876543227,'Coimbatore','05-APR-26');
1 row created.

SQL> INSERT INTO CUSTOMER VALUES('104','Gokul','gokul@gmail.com',9876543228,'Trichy','10-APR-26');
1 row created.

SQL> INSERT INTO CUSTOMER VALUES('105','Kishore','kishore@gmail.com',9876543229,'Chennai','15-JUL-26');
1 row created.

```

After inserting the records, the Customer table can be displayed to verify the stored information.

```

SQL> SELECT * FROM CUSTOMER;

```

CUSTOMER_ID	CUSTOMER_NAM	EMAIL	PHONE	ADDRESS	REGISTRATI
101	Pooja	pooja@gmail.com	9876543225	Madurai	15-MAR-26
102	Hari	hari@gmail.com	9876543226	Salem	01-APR-26
103	Lakshmi	lakshmi@gmail.com	9876543227	Coimbatore	05-APR-26
104	Gokul	gokul@gmail.com	9876543228	Trichy	10-APR-26
105	Kishore	kishore@gmail.com	9876543229	Chennai	15-JUL-26

Update:

The UPDATE statement is used to modify an existing customer's information. A particular customer can be selected using the customer_id, and the required field can then be changed.

```

SQL> UPDATE CUSTOMER SET PHONE = 9007654321 WHERE CUSTOMER_ID = '102';
1 row updated.

```

Conclusion

The Customer Database Module establishes the basic structure required for managing customer information in an e-commerce system. The identified entities and their relationships provide a foundation for connecting customers with orders, products, payments, and reviews. The Customer table operations demonstrate how customer records can be created, stored, viewed, and updated within the database.